Universidade Estadual do Piauí - UESPI
Curso de Tecnologia em Sistemas de Computação
Disciplina: Programação Web
Professor: Eyder Rios

2ª Avaliação
Trabalho de Implementação — Aplicação Web com React e Next.js
Problema 06 — Livro de Receitas
Data de entrega: 22/11/2026

PARTE I — REGRAS GERAIS

1. Objetivo

Desenvolver, em equipe de até **2 integrantes**, uma aplicação web de pequeno porte utilizando **React** e **Next.js**, composta por **backend** e **frontend**.

Cada equipe receberá um problema específico e uma API REST pública a ser utilizada.

A aplicação deverá envolver:

- componentes React e gerenciamento de estado;
- rotas e páginas com o App Router do Next.js;
- backend implementado com Route Handlers do Next.js;
- consumo de API REST externa pelo backend;
- persistência local com SQLite;
- comunicação entre frontend e backend via HTTP;
- operações assíncronas;
- responsividade e acessibilidade.

2. Tecnologias e restrições

Deverão ser utilizados:

- Next.js com **App Router**;
- React;
- **TypeScript**, obrigatoriamente em modo estrito (`"strict": true` no `tsconfig.json`);
- CSS e/ou Tailwind CSS;
- SQLite, por meio da biblioteca `better-sqlite3` ;
- `fetch` nativo ou biblioteca cliente HTTP (por exemplo, Axios).

Não poderão ser utilizados:

- Vue, Angular, Svelte ou frameworks equivalentes;
- servidor backend separado do Next.js (Express, NestJS, Fastify ou equivalentes);
- Server Actions;
- ORMs ou query builders (Prisma, Drizzle, Sequelize, Knex ou equivalentes);
- bancos de dados em nuvem, Firebase, Supabase ou serviços equivalentes;
- autenticação de usuários.

O código não deverá conter erros de compilação do TypeScript. O uso do tipo `any` deverá ser evitado e, quando indispensável, justificado em comentário.

A aplicação deverá utilizar efetivamente os dados fornecidos pela API definida no problema.

2.1 Por que Server Actions não poderão ser utilizadas

Server Actions são funções executadas no servidor que podem ser chamadas diretamente a partir de componentes React, sem que o desenvolvedor escreva a requisição HTTP nem o endpoint correspondente. O Next.js gera essa comunicação de forma implícita.

Neste trabalho, um dos objetivos é que a equipe **projete, implemente e explique a comunicação entre frontend e backend**: definição de rotas, métodos HTTP, parâmetros, formato das respostas e códigos de status. Com Server Actions, essa comunicação fica oculta pelo framework, o que impediria avaliar esses conteúdos.

Por isso, toda interação entre frontend e backend deverá ocorrer por meio de requisições HTTP explícitas aos endpoints em `app/api/` .

3. Arquitetura da aplicação

A aplicação deverá ser organizada em duas camadas, ambas no mesmo projeto Next.js.

3.1 Backend

O backend deverá ser implementado com **Route Handlers** em `app/api/`.

O backend será a **única** camada autorizada a:

- acessar a API externa;
- acessar o banco de dados SQLite.

Os endpoints do backend deverão:

- validar os parâmetros recebidos;
- retornar respostas em JSON;
- converter os dados da API externa em um formato próprio da aplicação, contendo apenas os campos necessários;
- realizar o processamento de dados exigido no problema específico;
- utilizar códigos de status HTTP adequados.

Códigos de status mínimos esperados:

Situação	Código
Consulta realizada com sucesso	`200`
Registro criado	`201`
Registro removido	`204`
Parâmetro ausente ou inválido	`400`
Recurso inexistente	`404`
Registro duplicado ou limite excedido	`409`
Falha na comunicação com a API externa	`502`
Erro interno	`500`

As respostas de erro deverão seguir um formato único, por exemplo:

```
{ "error": "Mensagem descritiva em português" }
```

3.2 Frontend

O frontend deverá ser implementado com componentes React e páginas do App Router.

O frontend deverá obter **todos** os dados exclusivamente por meio de requisições HTTP (com `fetch` ou biblioteca cliente HTTP) aos endpoints do próprio backend (`/api/...`).

Não será permitido que componentes acessem diretamente a API externa ou o banco de dados.

O frontend deverá possuir, no mínimo:

- página de listagem e/ou busca;
- página de detalhe utilizando **rota dinâmica** (por exemplo, `app/items/[id]/page.tsx`);
- página ou seção para gerenciamento dos dados persistidos;
- componentes reutilizáveis que recebam dados por `props`;
- navegação entre páginas com o componente `Link`.

4. Consumo da API externa

As requisições à API externa deverão ocorrer em tempo de execução, a partir do backend.

Não será permitido substituir a API por dados estáticos, copiados ou simulados quando ela estiver disponível.

A interface deverá tratar, quando aplicável:

- carregamento;
- sucesso;
- ausência de resultados;

- entradas inválidas;
- erros de comunicação ou processamento.

O usuário deverá receber feedback adequado durante operações assíncronas.

5. Persistência de dados

A aplicação deverá persistir localmente, em banco **SQLite**, os dados criados pelo usuário definidos no problema específico.

O banco de dados deverá:

- ser armazenado em arquivo local no projeto (por exemplo, `data/app.db`);
- ser criado automaticamente pela aplicação, caso o arquivo não exista;
- **não** ser incluído no repositório (o arquivo do banco deverá constar no `.gitignore`);
- operar **obrigatoriamente com o modo WAL desativado**, mantendo o modo de *journal* padrão do SQLite (`DELETE`). Não deverá ser executado o comando `PRAGMA journal_mode = WAL`.

Com o modo WAL desativado, todo o conteúdo do banco permanece em um único arquivo, sem os arquivos auxiliares `*.db-wal` e `*.db-shm`. Isso simplifica a configuração e garante que a remoção desse único arquivo restaure o banco ao estado inicial.

O backend deverá implementar as operações de **criação, consulta, atualização e remoção** (CRUD) da entidade definida no problema, utilizando os métodos HTTP correspondentes (`GET`, `POST`, `PUT` ou `PATCH`, `DELETE`).

Todas as consultas SQL deverão utilizar **parâmetros** (*prepared statements*). Não será permitido montar comandos SQL por concatenação de valores recebidos do usuário.

5.1 Registros iniciais

Ao criar o banco de dados, a aplicação deverá criar as tabelas e **inserir automaticamente registros iniciais**, de modo que todas as funcionalidades possam ser demonstradas já na primeira execução, sem que o avaliador precise cadastrar dados manualmente.

Os registros iniciais deverão:

- ser, no mínimo, **3** e, no máximo, **10**;
- referenciar identificadores **válidos e existentes** na API externa;
- apresentar valores variados, incluindo, quando aplicável, campos opcionais preenchidos e vazios;
- ser inseridos **somente** quando o banco for criado, sem duplicação em execuções posteriores ou recarregamentos do servidor de desenvolvimento;
- estar definidos em um único local do código (por exemplo, `lib/db.ts`).

Para restaurar o banco ao estado inicial, basta remover o arquivo do banco e reiniciar a aplicação.

5.2 Dados da API externa e dados do banco

O banco de dados deverá armazenar apenas os **dados criados pelo usuário** (por exemplo, favoritos, anotações, avaliações ou listas). Os dados fornecidos pela API externa **não deverão ser copiados integralmente** para o banco.

Cada fonte tem um papel distinto:

- a **API externa** é a fonte dos dados de consulta, que pertencem a terceiros e podem mudar a qualquer momento;
- o **banco local** guarda apenas o que o usuário produziu, referenciando os itens da API por seus identificadores.

É permitido armazenar, junto com o identificador, os poucos campos necessários para exibir uma listagem resumida (por exemplo, nome e URL da imagem), evitando uma requisição externa para cada item listado.

Essa regra existe por três motivos:

1. **Objetivo do trabalho.** Se os dados da API forem importados para o banco, a aplicação passa a funcionar sem consumir a API em tempo de execução, o que contraria a seção 4.
2. **Consistência.** Uma cópia local fica desatualizada em relação à fonte original.
3. **Boa prática.** Armazenar apenas o necessário reduz o volume de dados e evita duplicar informações de terceiros.

6. Interface e qualidade

A aplicação deverá:

- funcionar em diferentes tamanhos de tela;
- utilizar HTML semântico;
- possuir formulários e controles acessíveis;

- apresentar foco visível e contraste adequado;
- evitar sobreposição, textos cortados e rolagem horizontal desnecessária;
- manter código simples, legível e organizado.

Os identificadores técnicos deverão estar em **inglês**. Os textos da interface e da documentação deverão estar em **português**.

7. Equipe e repositório

O trabalho deverá ser realizado em equipes de até **2 integrantes**.

Todos os integrantes deverão conhecer toda a solução, incluindo backend e frontend.

O projeto deverá estar em um **repositório público no GitHub**, contendo:

- branch principal `main`;
- histórico de commits compatível com o desenvolvimento;
- participação efetiva dos integrantes;
- `.gitignore` (incluindo, no mínimo, `node_modules`, `.next` e o arquivo do banco de dados);
- `tsconfig.json` com `"strict": true`;
- `README.md`.

O `README.md` deverá informar:

- integrantes;
- nome e descrição do projeto;
- principais funcionalidades;
- API externa utilizada;
- endpoints do backend, com método HTTP e finalidade de cada um;
- estrutura da tabela do banco de dados, localização do arquivo e descrição dos registros iniciais;
- tecnologias;
- versão do Node.js utilizada;
- instruções de execução.

A aplicação deverá ser executável localmente apenas com os comandos:

```
npm install
npm run dev
```

Na primeira execução, o banco de dados deverá ser criado automaticamente com os registros iniciais.

Não é exigida a publicação (*deploy*) da aplicação.

8. Uso de inteligência artificial

É permitido utilizar ferramentas de IA como apoio.

A equipe continua responsável por todo o código entregue e deverá ser capaz de explicar seu funcionamento, incluindo componentes e estado em React, rotas do Next.js, endpoints do backend, requisições HTTP, acesso ao banco de dados, assincronismo e tratamento de erros.

9. Entrega e avaliação

A entrega será realizada pelo **SIGAA**, informando:

1. link do repositório público;
2. nomes dos integrantes.

A avaliação considerará:

- atendimento aos requisitos do problema;
- funcionamento da aplicação;
- organização e separação entre backend e frontend;
- integração com a API externa;
- persistência de dados e operações CRUD;
- tratamento de erros, validação e assincronismo;
- uso adequado de componentes, estado e rotas;
- responsividade e acessibilidade;

- organização do código e do repositório;
- documentação;
- domínio individual da solução.

Após a entrega, qualquer integrante poderá ser solicitado a explicar partes do código, demonstrar funcionalidades ou realizar pequenas alterações.

Funcionalidades adicionais não compensam requisitos obrigatórios não atendidos.

10. Checklist de entrega

Repositório:

- [ ] Repositório público no GitHub, na branch `main`.
- [ ] Commits de todos os integrantes.
- [ ] `.gitignore` com `node_modules`, `.next` e o arquivo do banco de dados.
- [ ] `tsconfig.json` com `"strict": true`.

Execução:

- [ ] Aplicação executada com `npm install` e `npm run dev`.
- [ ] Ausência de erros de compilação TypeScript.
- [ ] Banco de dados criado automaticamente com 3 a 10 registros iniciais.
- [ ] Modo WAL desativado.

Backend:

- [ ] Todos os endpoints mínimos do problema implementados.
- [ ] Validação de parâmetros.
- [ ] Códigos HTTP da tabela da seção 3.1, incluindo `409`.
- [ ] Erros em JSON no formato padronizado.
- [ ] SQL parametrizado.
- [ ] Cabeçalho de identificação enviado à API externa, quando exigido.

Frontend:

- [ ] Páginas mínimas do problema.
- [ ] Rota dinâmica de detalhe.
- [ ] Dados obtidos somente via `/api/...`.
- [ ] Estados de carregamento, vazio e erro.
- [ ] Responsividade e acessibilidade.

Documentação:

- [ ] `README.md` com todos os itens da seção 7, incluindo endpoints, tabela do banco de dados e registros iniciais.

Entrega:

- [ ] Link do repositório e nomes dos integrantes enviados pelo SIGAA até 22/11/2026.

11. Orientação final

Prefira uma solução **simples, correta, organizada, acessível e compreendida pela equipe**.

Não é necessário implementar funcionalidades além das exigidas no problema específico.

12. Requisitos complementares

Os requisitos abaixo complementam as seções anteriores.

Endpoints da entidade persistida

O problema define uma entidade a ser persistida no SQLite. Seus endpoints deverão seguir o padrão:

Método	Rota	Finalidade
`GET`	`/api/<recurso>`	Listar os registros
`POST`	`/api/<recurso>`	Criar um registro
`PATCH`	`/api/<recurso>/[id]`	Atualizar os campos editáveis de um registro
`DELETE`	`/api/<recurso>/[id]`	Remover um registro

Regras:

- o `id` da rota é o identificador do registro no banco local, e não o identificador da API externa;
- um mesmo item da API externa não poderá ser cadastrado duas vezes; nesse caso, o backend deverá retornar `409` ;
- o `PATCH` deverá aceitar apenas os campos indicados como **editáveis**;
- campos editáveis com valor inválido deverão resultar em `400` ;
- o campo `created_at` deverá ser preenchido automaticamente pelo backend.

Rotas e nomes

- As rotas e os nomes de campos indicados são os **mínimos esperados**. Podem ser acrescentados outros, desde que mantidos em inglês e documentados no `README.md` .
- Os parâmetros de consulta dos endpoints do backend são definidos pela equipe, conforme o problema, e também deverão ser documentados.
- As rotas e os parâmetros da **API externa** não são fornecidos neste enunciado. Consultar a documentação da API faz parte do trabalho.

Identificação junto à API externa

Algumas APIs exigem ou recomendam que a aplicação se identifique por meio de um cabeçalho HTTP. Esse cabeçalho deverá ser enviado pelo backend, conforme indicado na seção 13.

Indisponibilidade da API externa

APIs públicas podem mudar ou ficar temporariamente indisponíveis. Em caso de indisponibilidade prolongada ou de alteração que impeça o desenvolvimento, a equipe deverá comunicar o professor imediatamente, sem substituir a API por dados simulados.

PARTE II — PROBLEMA ESPECÍFICO

13. Problema — Livro de Receitas

API: TheMealDB
Base: `https://www.themealdb.com/api/json/v1/1`
Documentação: https://www.themealdb.com/api.php

Descrição

Aplicação que permite buscar receitas por nome, categoria ou culinária de origem, consultar ingredientes e modo de preparo e manter um caderno de receitas avaliadas pelo usuário.

Backend — endpoints mínimos

Método	Rota	Finalidade
`GET`	`/api/meals`	Buscar receitas por nome, categoria **ou** culinária
`GET`	`/api/meals/[id]`	Retornar os detalhes de uma receita
`GET`	`/api/filters`	Listar as categorias e culinárias disponíveis
`GET POST`	`/api/cookbook`	Listar e incluir receitas no caderno
`PATCH DELETE`	`/api/cookbook/[id]`	Atualizar e remover receitas do caderno

Processamento exigido no backend

- **Unificar** as diferentes formas de busca da API externa em um único endpoint, com formato de resposta idêntico para os três casos. Exatamente um critério deverá ser informado; caso contrário, retornar `400` .
- Converter os campos numerados de ingredientes e medidas da API em uma **lista de objetos** (ingrediente e medida), descartando os campos vazios.
- Dividir o modo de preparo em uma lista de passos.

Persistência — entidade `cookbook`

Campo	Descrição	Editável
`id`	Identificador local	—
`meal_id`	Identificador da receita na API	Não

Campo	Descrição	Editável
`name`	Nome da receita	Não
`image_url`	URL da imagem	Não
`rating`	Avaliação (inteiro de 1 a 5)	Sim
`note`	Anotação (opcional, até 500 caracteres)	Sim
`created_at`	Data de inclusão	—

Frontend — páginas mínimas

- Busca de receitas por nome, categoria ou culinária.
- Detalhe da receita (`/meals/[id]`), com imagem, lista de ingredientes e passos do preparo.
- Página do caderno de receitas, com edição da avaliação e da anotação e remoção.

Observações

- A chave de teste `1` é destinada a uso educacional e de desenvolvimento.